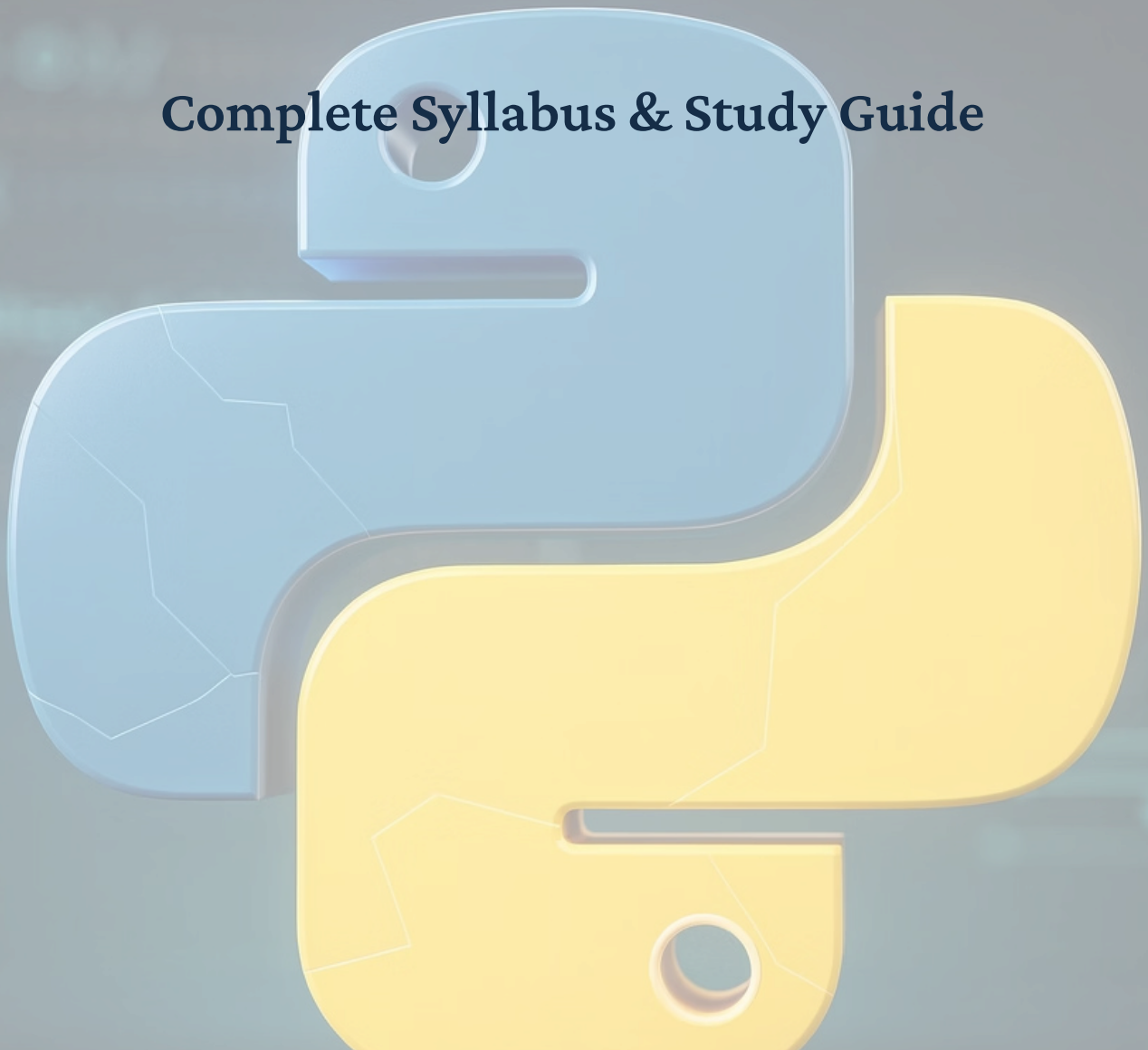


PYTHON

Complete Syllabus & Study Guide



BROWSEJOBS

Module 1: Introduction to Python

1.1 What is Python?

Python is a high-level, interpreted programming language created by Guido van Rossum. It is widely used in data engineering, data science, web development, automation, AI, and machine learning. For data professionals, Python serves as the foundation before learning SQL, PySpark, Pandas, AWS, and Azure.

Key Features of Python

Sub-Topic	Explanation	Code Example
English-like Syntax	About 40% of Python reads like English, making it one of the easiest programming languages to learn.	<code>print('Hello World')</code>
Interpreted	Python executes code line by line (no compilation step needed). Errors are caught at runtime.	<code># Run with: python file.py</code>
Dynamically Typed	You do not need to declare variable types. Python figures out the type automatically.	<code>x = 10 # int</code> <code>x = 'hi' # now str</code>
Large Community	Massive ecosystem of libraries and active community support for debugging and development.	<code>pip install pandas</code>
Cross-Platform	Python runs on Windows, macOS, and Linux without modification.	<code># Same code everywhere</code>
Object-Oriented	Supports OOP concepts: classes, objects, inheritance, encapsulation, and polymorphism.	<code>class Car: pass</code>



Real-World Scenario: Why Python for Data Engineering?

At companies like Goldman Sachs, Netflix, and Infosys, data engineers use Python daily to build ETL pipelines, automate data processing, and integrate with cloud services. A data engineer might write a Python script to extract data from APIs, transform it using Pandas, and load it into AWS S3 or Azure Data Lake.

1.2 Running Python Code

In Google Colab (the environment used in this course), you execute code cells using **Shift+Enter**. Each cell runs independently, and the output appears below the cell immediately.

1.3 How Computers Understand Python

Computers only understand binary (0s and 1s). The Python interpreter translates your human-readable code into machine-executable instructions. When you write `print('krish')`, the interpreter converts this into binary operations that display text on your screen.

Module 2: Variables & Naming Conventions

2.1 What are Variables?

Variables are containers that store data values. Think of them as labeled boxes where you keep information. Unlike languages like Java or C++, Python does not require you to declare the type of a variable.

```
first_name = 'krish' # string variable
age = 16           # integer variable
is_student = True  # boolean variable
```

2.2 Variable Naming Rules

Sub-Topic	Explanation	Code Example
Letters & Numbers	Variables can contain a-z, A-Z, 0-9, and underscore (_)	<code>my_var = 10</code> <code>var123 = 20</code>
Cannot Start with Number	Variable names must begin with a letter or underscore	<code>123var = 5</code> # ERROR <code>_var = 5</code> # OK
Case Sensitive	x and X are two completely different variables in Python	<code>x = 1</code> <code>X = 2</code> # different!
No Keywords	You cannot use Python reserved words like print, if, for, class, etc.	<code>print = 5</code> # BAD <code>my_print = 5</code> # OK
No Special Chars	Only underscore is allowed. No %, &, @, #, etc.	<code>%myvar</code> # ERROR <code>my_var</code> # OK

2.3 PEP8 Standards (Style Guide)

PEP8 is Python's official style guide that ensures code readability and consistency across teams. Following PEP8 is essential in professional environments.

Rule 1
Limit all lines to **79 characters** for improved readability.

Rule 2
Use **lowercase with underscores** for variable and function names (snake_case).
`first_name = 'krish'` # Good (PEP8 compliant)
`firstName = 'krish'` # Bad (camelCase not recommended)

Rule 3
Use **proper indentation** (4 spaces per level). Python enforces this with colons (:).

Rule 4
Use **docstrings and comments** to describe functions and modules.

2.4 Comments & Documentation

Sub-Topic	Explanation	Code Example
Single-line Comment	Use # to add a comment. Python ignores everything after #.	<code># This prints hello</code> <code>print('hello')</code>
Multi-line Docstring	Use triple quotes (""" or """) for multi-line comments or documentation strings.	<code>""" This function calculates</code> <code>area """</code>
Keyboard Shortcut	Use Ctrl + / to toggle comments on selected lines in most editors.	<code># Select lines + Ctrl+/</code>



Real-World Scenario: Why PEP8 Matters
When working at companies like HCL or Infosys on data engineering projects, your code is reviewed by senior engineers. Non-PEP8 code gets rejected in code reviews. Consistent naming like `customer_transaction_date` instead of `ctd` makes code maintainable across teams of 10-50 engineers.



Module 3: Data Types

3.1 Integer (int)

Whole numbers (positive, negative, or zero) without decimal points. Used for counting, indexing, and mathematical operations.

```
age = 25
temperature = -10
count = 0
```

3.2 Float (float)

Numbers with decimal points. Behave exactly like integers in arithmetic operations but include precision.

```
price = 1.01 + 1.04 # Result: 2.05
salary = 50000.75
```

3.3 Boolean (bool)

Represents True (1) or False (0). The T and F must be capitalized. Booleans are the backbone of all conditional logic.

```
is_active = True
5 > 3      # True
True + True # 2 (treated as 1+1)
```

3.4 String (str)

Text data enclosed in single quotes ('...') or double quotes ("..."). Everything inside quotes is a string, even numbers.

```
name = 'krish'
num_str = '123'    # This is a string, not a number
y = "bangalore's"  # Use double quotes for apostrophes
```

3.5 Arithmetic Operators

Sub-Topic	Explanation	Code Example
Addition (+)	Adds two numbers together	<code>10 + 5 # 15</code>
Subtraction (-)	Subtracts right from left	<code>10 - 2 # 8</code>
Multiplication (*)	Multiplies two numbers	<code>10 * 2 # 20</code>
Division (/)	Divides and returns float	<code>10 / 2 # 5.0</code>
Modulus (%)	Returns the remainder after division. Used to check even/odd.	<code>10 % 2 # 0 (even)</code> <code>11 % 2 # 1 (odd)</code>
Floor Division (//)	Divides and rounds down to nearest integer. Used to find midpoints.	<code>15 // 2 # 7</code> <code>11 // 2 # 5</code>
Exponent (**)	Raises to the power of	<code>2 ** 4 # 16</code>

3.6 Type Casting

Converting one data type to another. Essential when reading user input (always string) or processing data from files and APIs.

Sub-Topic	Explanation	Code Example
<code>int()</code>	Converts to integer. Truncates decimals.	<code>int(10.66) # 10</code> <code>int('10') # 10</code>
<code>float()</code>	Converts to floating point number.	<code>float(6) # 6.0</code>
<code>str()</code>	Converts to string.	<code>str(10) # '10'</code>
<code>bool()</code>	Converts to boolean. 0 is False, everything else is True.	<code>bool(1) # True</code> <code>bool(0) # False</code>

Module 4: String Operations & Methods

4.1 String Indexing & Slicing



```
x = 'krishna'
x[0] # 'k' (first character)
x[-1] # 'a' (last character)
x[0:3] # 'kri' (index 0, 1, 2)
x[::2] # 'krsa' (every 2nd character)
x[::-1] # 'anhsirk' (reversed string)
```

4.2 String Methods

Sub-Topic	Explanation	Code Example
upper() / lower()	Converts entire string to uppercase or lowercase.	'hello'.upper() # 'HELLO' 'HELLO'.lower() # 'hello'
strip()	Removes whitespace (or specified chars) from both ends.	' krish '.strip() # 'krish'
replace()	Replaces occurrences of a substring with another.	'hello'.replace('l','k') # 'hekko'
capitalize()	Capitalizes only the first letter.	'krishna bhargav'.capitalize()
startswith() / endswith()	Checks if string starts/ends with a given value. Returns boolean.	'anil'.startswith('a') # True
find()	Returns the index of the first occurrence of a substring.	'bangalore'.find('gal') # 3
isalpha()	Returns True if all characters are alphabets (no spaces or numbers).	'krish'.isalpha() # True
isdecimal()	Returns True if all characters are digits 0-9.	'12345'.isdecimal() # True
split()	Splits string into a list using a delimiter.	'my,name,is'.split(',') # ['my','name','is']
join()	Joins a list into a string using a separator.	' '.join(['hello','world']) # 'hello world'



4.3 String Concatenation

Joining strings together using the `+` operator. You can only concatenate string with string (not string with integer directly).

```
'krish' + ' ' + 'bhargav' # 'krish bhargav'
'1' + 'krish'           # '1krish' (string, not addition)
```

4.4 Format Strings (f-strings)

F-strings allow you to embed variables directly inside strings. Prefix the string with `f` and place variables in curly braces.

```
name = 'raj'
print(f'welcome to icici, {name}') # welcome to icici, raj
```



Real-World Scenario: String Processing in Data Pipelines

Data engineers constantly clean and transform string data. For example, cleaning customer names from a database: stripping extra spaces, standardizing to lowercase, splitting full names into first/last, and validating email formats using string methods before loading into a data warehouse.

Module 5: Operators

5.1 Assignment Operators

Used to assign and update values. Shorthand operators combine arithmetic with assignment.

```
x = 10
x += 5 # x = x + 5 = 15
x *= 2 # x = x * 2 = 30
x -= 10 # x = x - 10 = 20
x /= 4 # x = x / 4 = 5.0
```

5.2 Comparison Operators

Sub-Topic	Explanation	Code Example
<code>==</code> (Equal)	Checks if two values are equal	<code>10 == 10 # True</code>
<code>!=</code> (Not Equal)	Checks if two values are not equal	<code>5 != 6 # True</code>
<code>></code> (Greater Than)	Checks if left is greater than right	<code>10 > 5 # True</code>
<code><</code> (Less Than)	Checks if left is less than right	<code>3 < 7 # True</code>
<code>>=</code> / <code><=</code>	Greater/less than or equal to	<code>5 <= 5 # True</code>

5.3 Logical Operators

Used to combine conditional statements. Critical for building complex business logic.

Sub-Topic	Explanation	Code Example
<code>and</code>	Both conditions must be True	<code>x>5 and y<10 # both checked</code>
<code>or</code>	At least one condition must be True	<code>x>5 or y<10 # either works</code>
<code>not</code>	Reverses the boolean output	<code>not(True) # False</code>

```
x = 10; y = 20
x >= 10 and y == 20    # True (both True)
not(x < 5 or y == 6)   # True
```

5.4 Identity Operators

`is` and `is not` check whether two variables point to the same object in memory (not just equal values).

```
x = 6; y = x
x is y # True (same object in memory)
```



Real-World Scenario: Logical Operators in ETL
In data pipelines, you might filter records like: `if status == 'active' and last_login > threshold and region in allowed_regions`. Banking systems use these to validate transactions: `if amount > 0 and balance >= amount and not is_frozen`.

Module 6: Data Structures

Lists []

Ordered, allows duplicates, **mutable**, heterogeneous (mixed types).

Tuples ()

Ordered, allows duplicates, **immutable** (cannot be changed), heterogeneous.

Dictionaries { key: value }

Ordered, no duplicate keys, **mutable**. Stores data as key-value pairs (like JSON).

Sets { }

Unordered, **no duplicates**, slightly mutable. Cannot be indexed.

6.1 Lists []

```
fruits = ['apple', 'banana', 'mango', 'apple']
numbers = [1, 2, 3, 4, 5]
mixed = [1, 'krish', True, 1.004]
```

Sub-Topic	Explanation	Code Example
Indexing / Slicing	Access elements by position. Same rules as strings.	<code>x[0]</code> <code>x[-1]</code> <code>x[1:3]</code> <code>x[::-1]</code>
<code>append()</code>	Adds a single element to the end of the list.	<code>x.append(11)</code>
<code>extend()</code>	Adds all elements of another list to the end.	<code>x.extend([4,5,6])</code>
<code>insert()</code>	Inserts element at a specific position.	<code>x.insert(1, 'orange')</code>
<code>remove()</code>	Removes the first occurrence of a value (not by index).	<code>x.remove('apple')</code>
<code>pop()</code>	Removes by index (default: last element). Returns the removed item.	<code>x.pop()</code> # removes last <code>x.pop(0)</code> # removes first
<code>clear()</code>	Removes all elements from the list.	<code>x.clear()</code> # <code>x = []</code>
<code>del</code>	Deletes the entire list variable from memory.	<code>del x</code>

6.2 Tuples ()

Tuples are used when you want to protect data from being modified. To modify, convert to list first, make changes, then convert back.

```
x = (1, 2, 3, 4, 5)
# x[0] = 10 # ERROR! Tuples are immutable
y = list(x) # Convert to list
y.append(10) # Modify the list
x = tuple(y) # Convert back to tuple
```

6.3 Dictionaries { key: value }

```
student = {
    'fname': 'krishna',
    'lname': 'bhargav',
    'age': 25,
    'city': 'bangalore',
    'phonenumbers': [123456789, 987654321]
}
```

Sub-Topic	Explanation	Code Example
Access Value	Use the key to get its value.	<code>student['fname']</code> # 'krishna'
Add/Update	Assign a value to a key. Creates if new, updates if exists.	<code>student['email'] = 'a@b.com'</code>
<code>keys()</code>	Returns all keys.	<code>student.keys()</code>
<code>values()</code>	Returns all values.	<code>student.values()</code>
<code>items()</code>	Returns all key-value pairs as tuples.	<code>student.items()</code>
<code>pop(key)</code>	Removes a specific key-value pair.	<code>student.pop('email')</code>
<code>popitem()</code>	Removes the last inserted key-value pair.	<code>student.popitem()</code>

Nested Dictionaries

Dictionaries can contain other dictionaries, lists, and complex structures. This is exactly how JSON data from APIs is structured.

```
employees = {
  'IT': {
    'emp1': {'fname':'krishna', 'age':25},
    'emp2': {'fname':'ajay', 'age':26}
  }
}
employees['IT']['emp1']['fname'] # 'krishna'
```

6.4 Sets { }

```
myset = {'apple', 'banana', 'mango', 'apple'}
print(myset) # {'apple', 'banana', 'mango'} - duplicate removed!
# Remove duplicates from a list:
i = [1,2,3,4,4,4,5,1,2]
unique = list(set(i)) # [1, 2, 3, 4, 5]
```

Sub-Topic	Explanation	Code Example
add()	Adds a single element.	myset.add('grapes')
update()	Adds multiple elements from a list.	myset.update(['a','b'])
remove()	Removes a specific element.	myset.remove('apple')
union()	Combines two sets (no duplicates).	A.union(B)

 **Real-World Scenario: Data Structures in Production**

Lists are used to store batches of records. Dictionaries mirror JSON payloads from REST APIs. Sets are used for deduplication in data pipelines. At Goldman Sachs, trading data arrives as nested JSON (dictionaries) that engineers parse to extract specific fields like `order["trade_details"]["instrument"]["price"]`.

List vs. Tuple vs. Set vs. Dictionary

Python offers four fundamental built-in data structures, each optimized for different use cases. Understanding their core differences—particularly regarding mutability, order, and how they handle duplicate elements—is crucial for efficient and robust programming. Choosing the right data structure can significantly impact your code's performance and readability, making this comparison a cornerstone for any Python developer.

Here's a breakdown of their key characteristics:

Feature	List <code>[]</code>	Tuple <code>()</code>	Set <code>{}</code>	Dictionary <code>{key:value}</code>
Ordered	Yes (insertion order)	Yes (insertion order)	No (unordered collection)	Yes (insertion order from Python 3.7+)
Mutable	Yes	No (immutable)	Yes (can add/remove elements)	Yes
Allows Duplicates	Yes	Yes	No (unique elements only)	No (unique keys; values can be duplicated)
Access/Indexing	By integer index	By integer index	No (unordered)	By key
Typical Use Case	Collections of items where order matters and items may change.	Fixed collections of items; function return values (e.g., coordinates).	Storing unique items; membership testing; mathematical set operations.	Storing data as key-value pairs; representing structured records (like JSON).

Understanding these distinctions is paramount for effective data handling in Python. For instance, if you need a collection of items that should never be altered, a tuple is the appropriate choice. If you need to quickly check for unique items or perform set operations, a set is ideal. For structured data mapping, dictionaries are indispensable.



Module 7: Loops (for & while)

7.1 For Loop

A for loop iterates over a sequence (list, tuple, string, range) a fixed number of times. It determines how many iterations are needed and then executes that many times.

```
fruits = ['apple', 'banana', 'mango']
for item in fruits:
    print(item) # Output: apple, banana, mango
```

```
# Sum of numbers 1 to 10
total = 0
for i in range(1, 11):
    total += i
print(total) # 55
```

```
# Print odd numbers from 1 to 10
for i in range(1, 11):
    if i % 2 != 0:
        print(i) # 1, 3, 5, 7, 9
```

7.2 While Loop

A while loop continues executing as long as a condition remains True. Use when you don't know the exact number of iterations in advance.

```
count = 1
while count <= 5:
    print('krish')
    count += 1
```

Password Validation Example



```
correct = 'admin123'
attempts = 0
max_attempts = 3
while attempts < max_attempts:
    entered = input('Enter Password: ')
    if entered == correct:
        print('Password accepted!')
        break
    else:
        attempts += 1
        remaining = max_attempts - attempts
        print(f'{remaining} attempts left')
```

7.3 Loop Control: break, continue, pass

break

Exits the loop entirely.

```
if x == 5: break
```

continue

Skips the current iteration and moves to the next.

```
if x == 3: continue
```

pass

Does nothing. Used as a placeholder.

```
if x > 10: pass
```

Real-World Scenario: Loops in Data Processing

Data engineers use for loops to iterate through millions of database records, apply transformations, and write to output. While loops are used in retry mechanisms: `while not connected and retries < 5: try_connection()`. A log monitoring system might use `while True` to continuously read and process new log entries.



Module 8: Comprehensions

8.1 List Comprehension

A concise, one-line way to create new lists. Follow the **OAC pattern**: Output, Action, Condition.

```
# Syntax: [Output for Action if Condition]
x = [1, 2, 3, 4, 5, 6]
squares = [i*i for i in x]          # [1, 4, 9, 16, 25, 36]
evens = [i for i in x if i%2 == 0] # [2, 4, 6]
even_sq = [i*i for i in x if i%2==0] # [4, 16, 36]
```

Flattening Nested Lists

```
nested = [[1,2,3], [4,5,6], [7,8,9]]
flat = [j for i in nested for j in i]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

8.2 Dictionary Comprehension

```
# {key: value for action if condition}
squares = {i: i*i for i in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

words = ['data', 'engineering', 'is', 'hot']
lengths = {w: len(w) for w in words}
# {'data': 4, 'engineering': 11, 'is': 2, 'hot': 3}
```

8.3 Set Comprehension

```
unique_squares = {i*i for i in range(10)}
# {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
```

❏ Real-World Scenario: Comprehensions in ETL

When processing a CSV with 100K rows, list comprehensions are significantly faster than traditional loops. Example: `valid_emails = [row['email'] for row in data if '@' in row['email'] and row['status'] == 'active']`



Module 9: Functions

9.1 Defining & Calling Functions

Functions are reusable blocks of code that perform a specific task. They reduce code duplication and improve organization. Defined with the `def` keyword.

```
def greet(customer_name):  
    print(f'Hello {customer_name}, welcome to ICICI!')  
  
greet('krish') # Hello krish, welcome to ICICI!
```

9.2 Parameters & Default Values

```
def printname(age, name='krish'): # name has a default value  
    print(age, name)  
  
printname(16, 'ajay') # 16 ajay  
printname(16)        # 16 krish (uses default)
```

9.3 Return vs Print

`print()` displays output to screen. `return` sends a value back to the caller so it can be stored or used further.

```
def square(x):  
    return x * x  
  
result = square(12) # result = 144 (stored for later use)
```



9.4 Practical Function Examples

Phone Number Validator

```
def
phone_validator(numb
er):
    if number.isdigit()
and len(number)==10
and number[0]=='9':
        print('Valid')
    else:
        print('Invalid
phone number')
```

Count Vowels

```
def count_vowels(s):
    vowels =
'aeiouAEIOU'
    count = 0
    for char in s:
        if char in vowels:
            count += 1
    return count

count_vowels('bangalor
e') # 4
```

Fibonacci Sequence

```
def fibonacci(n):
    a, b = 0, 1
    fib = []
    for i in range(n):
        fib.append(a)
        a, b = b, a + b
    return fib

fibonacci(10) #
[0,1,1,2,3,5,8,13,21,34]
```

Module 10: *args, **kwargs & Lambda Functions

10.1 *args (Arbitrary Arguments)

When you don't know how many arguments will be passed, use `*args`. It collects all positional arguments into a tuple.

```
def add_numbers(*args):
    total = 0
    for i in args:
        total += i
    return total

add_numbers(1, 2, 3, 4, 5) # 15
add_numbers(10, 20)      # 30
```

10.2 **kwargs (Keyword Arguments)

Collects all keyword arguments into a dictionary. Useful for configuration-style functions.

```
def print_details(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

print_details(name='krish', age=16, gender='m')
```

10.3 Lambda Functions

Anonymous (unnamed), one-time-use functions. Written in a single line. Commonly used with `map()`, `filter()`, and `reduce()`.

```
# Syntax: lambda parameters: expression
square = lambda x: x * x
square(10) # 100

check_even = lambda x: 'Even' if x%2==0 else 'Odd'
check_even(10) # 'Even'
```



10.4 map(), filter(), reduce()

Sub-Topic	Explanation	Code Example
map()	Applies a function to every element. Returns transformed list.	<code>list(map(lambda x: x*x, [1,2,3])) # [1,4,9]</code>
filter()	Filters elements that satisfy a condition. Returns matching items.	<code>list(filter(lambda x: x%2==0, [1,2,3,4])) # [2,4]</code>
reduce()	Reduces a list to a single value by applying function cumulatively.	<code>reduce(lambda x,y: x+y, [1,2,3,4,5]) # 15</code>



Real-World Scenario: Lambda in Data Transformation

In PySpark and Pandas, lambda functions are used extensively: `df['salary'].apply(lambda x: x * 1.1)` gives all employees a 10% raise. In ETL: `cleaned = list(map(lambda r: r.strip().lower(), raw_data))`.



Module 11: Sorting Algorithms

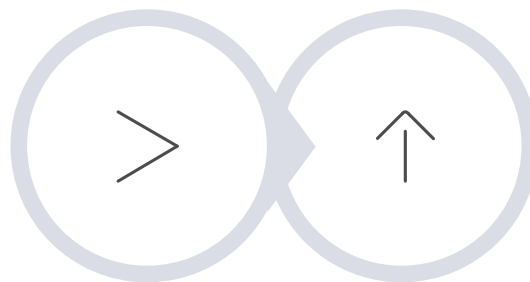
11.1 Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. The largest element 'bubbles up' to the end in each pass. Understanding this algorithm builds strong foundations for technical interviews.

```
def bubble_sort(x):  
    n = len(x)  
    for i in range(n):          # outer loop: passes  
        for j in range(n - i - 1): # inner loop: comparisons  
            if x[j] > x[j+1]:     # compare adjacent  
                x[j], x[j+1] = x[j+1], x[j] # swap  
    return x
```

```
bubble_sort([100, 32, 5, 6, 12, 9, 60])  
# [5, 6, 9, 12, 32, 60, 100]
```

The key insight: after each outer pass, one more element from the right is in its correct position, so the inner loop can reduce its range by i each time ($n-i-1$).



Pass 1

Pass 2

❏ Real-World Scenario: Why Learn Sorting?

While Python has built-in `sorted()` and `.sort()`, understanding sorting algorithms is essential for data engineering interviews at companies like Goldman Sachs, Amazon, and Google. It demonstrates algorithmic thinking and helps you understand time complexity ($O(n^2)$ for bubble sort vs $O(n \log n)$ for merge sort).

Module 12: Exception Handling

12.1 try / except / else / finally

Exception handling prevents your program from crashing when errors occur. Instead of breaking, it gracefully handles the error and continues.

Sub-Topic	Explanation	Code Example
try	The code that might cause an error goes here.	try: x = 10/0
except	Catches and handles specific errors.	except ZeroDivisionError: print('Cannot divide by 0')
else	Runs only if no exception occurred in try.	else: print('Success!')
finally	Always runs, regardless of error or not. Used for cleanup.	finally: print('Done')

```
try:
    file = open('data.csv', 'r')
    content = file.read()
except FileNotFoundError:
    print('File not found!')
else:
    print('File read successfully')
finally:
    print('Cleanup complete')
```

12.2 Raising Custom Exceptions

```
def withdraw(amount):
    balance = 1000
    if amount > balance:
        raise Exception('Insufficient balance!')
    return balance - amount

try:
    print(withdraw(10000))
except Exception as e:
    print('Error:', e) # Error: Insufficient balance!
```

 Real-World Scenario: Exception Handling in Production

Every production data pipeline must handle exceptions: API timeouts, file not found errors, database connection failures, malformed data. Without proper exception handling, a single bad record in 1 million can crash an entire ETL job. Engineers wrap critical operations in try/except to log errors and continue processing.

Module 13: File Handling & Packages

13.1 Reading & Writing Files

Sub-Topic	Explanation	Code Example
Read ('r')	Opens file for reading. Error if file doesn't exist.	<code>with open('f.txt','r') as f: print(f.read())</code>
Write ('w')	Opens file for writing. Creates if not exists. Overwrites if exists.	<code>with open('f.txt','w') as f: f.write('hello')</code>
Append ('a')	Opens file for appending. Adds to end without overwriting.	<code>with open('f.txt','a') as f: f.write('\nnew line')</code>
Delete	Use os module to delete files.	<code>import os os.remove('f.txt')</code>

13.2 Installing Packages with pip

`pip` is Python's package manager. It installs libraries from the Python Package Index (PyPI).

pandas <code>pip install pandas</code> Data manipulation	numpy <code>pip install numpy</code> Numerical computing	requests <code>pip install requests</code> HTTP/API calls
--	--	---

 Real-World Scenario: File Handling in ETL

Data engineers read CSV/JSON files from data lakes, process them, and write output files. The 'with' statement ensures files are properly closed even if an error occurs. In production, you might read 5GB log files line by line using generators to avoid memory overflow.

Module 14: Regular Expressions & JSON

14.1 Regex (Regular Expressions)

Regex is a powerful tool for finding patterns in text data. Used extensively in data validation, cleaning, and extraction.

Common Regex Patterns

Sub-Topic	Explanation	Code Example
<code>\d</code>	Matches any digit (0-9)	<code>\d{10}</code> matches phone numbers
<code>\w</code>	Matches word characters (a-z, 0-9, _)	<code>\w+</code> matches words
<code>\s</code>	Matches whitespace	<code>\s</code> matches spaces/tabs
<code>[]</code>	Character set	<code>[a-zA-Z]</code> matches letters
<code>^ \$</code>	Start and end of string	<code>^The.*Spain\$</code> full match
<code>* + ?</code>	Quantifiers: 0+, 1+, 0 or 1	<code>he.*o</code> matches heeeeo
<code>{n}</code>	Exactly n occurrences	<code>\d{6}</code> matches pincodes

Practical Regex Examples

```
import re
text = 'Krish born 16-05-1994, email: krish@gmail.com, phone: 6363402404'

# Extract email
emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', text)

# Extract phone (10 digits)
phones = re.findall(r'\b\d{10}\b', text)

# Extract date
dates = re.findall(r'\b\d{2}-\d{2}-\d{4}\b', text)
```




14.2 JSON (JavaScript Object Notation)

JSON is the standard data format for APIs. Python's `json` module converts between JSON strings and Python dictionaries.

```
import json

# String to Dictionary (loads)
x = '{"name":"krish","age":30}'
y = json.loads(x)    # y is now a dict
print(y['name'])     # 'krish'

# Dictionary to String (dumps)
data = {'name':'krish', 'age':30}
json_str = json.dumps(data) # '{"name":"krish","age":30}'
```



Real-World Scenario: Regex in Data Cleaning

When processing raw customer data, regex extracts structured information from unstructured text: phone numbers from support tickets, email addresses from web scraping, dates from legal documents, and postal codes from address fields. API responses are JSON, making `json.loads()` essential for every API integration.



Module 15: Decorators & Generators

15.1 Decorators

A decorator is a function that adds extra functionality to another function without modifying its code. It wraps around the original function using the `@` syntax.

```
def my_decorator(func):  
    def wrapper():  
        print('Before the function')  
        func()  
        print('After the function')  
    return wrapper
```

```
@my_decorator  
def say_hello():  
    print('Hello!')
```

```
say_hello()  
# Before the function  
# Hello!  
# After the function
```

Login Required Decorator

```
def login_required(func):  
    def wrapper(user):  
        if user['loggedin']:  
            func(user)  
        else:  
            print('Access denied!')  
    return wrapper
```

```
@login_required  
def view_profile(user):  
    print('Welcome to your profile!')
```



15.2 Generators

Generators are memory-efficient iterators that yield one value at a time using the `yield` keyword. Unlike `return` (which sends all data at once), `yield` pauses the function and resumes from where it left off.

```
def read_file(filepath):  
    with open(filepath, 'r') as file:  
        for line in file:  
            yield line # Processes one line at a time  
  
# Process a 5GB log file without loading it all into memory  
for line in read_file('huge_log.csv'):  
    print(line)
```



Real-World Scenario: Generators in Big Data

When processing 5GB+ log files or streaming data, loading everything into memory crashes the system. Generators process data one chunk at a time. Apache Airflow DAGs and PySpark use this concept (lazy evaluation) extensively. Decorators are used in web frameworks like Flask/Django for authentication and logging.

Module 16: Object-Oriented Programming (OOP)

16.1 Classes & Objects

A class is a blueprint for creating objects. An object is an instance of a class with its own data (attributes) and behaviors (methods). The `__init__` method is the constructor that initializes the object.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def drive(self):
        print(f'{self.brand} {self.model} is driving')

car1 = Car('Mercedes', 'C220D')
car1.drive() # Mercedes C220D is driving
```

16.2 Encapsulation

Wrapping data and methods into a single unit (class) and controlling access using public/private attributes. Private variables use double underscore (`__`) prefix.

```
class BankAccount:
    def __init__(self, acc_num, balance):
        self.__balance = balance # Private!
    def deposit(self, amount):
        self.__balance += amount
    def get_balance(self):
        return self.__balance
```

16.3 Abstraction

Hiding complex implementation details and exposing only what is necessary. Uses Abstract Base Classes (ABC) to define templates that child classes must implement.

```
from abc import ABC, abstractmethod
class Vehicle(ABC):
    @abstractmethod
    def start(self): pass
    @abstractmethod
    def stop(self): pass

class Car(Vehicle):
    def start(self): print('Car started')
    def stop(self): print('Car stopped')
```

16.4 Inheritance



Sub-Topic	Explanation	Code Example
Single	One child inherits from one parent.	<code>class Student(Human): pass</code>
Multiple	One child inherits from multiple parents.	<code>class Child(Mother, Father): pass</code>
Multilevel	Grandparent > Parent > Child chain.	<code>class Child(Parent): class Parent(Grand):</code>
Hierarchical	Multiple children inherit from same parent.	<code>class Child1(Parent): class Child2(Parent):</code>
Hybrid	Mix of multiple inheritance types.	<code>class D(B, C): class B(A): class C(A):</code>

16.5 Polymorphism

'Many forms' – the same method name behaves differently depending on the object that calls it.

```
class Vehicle:
    def start(self): print('Start vehicle')

class Car(Vehicle):
    def start(self): print('Start car') # Method Override

class Bike(Vehicle):
    def start(self): print('Start bike') # Method Override

for v in [Car(), Bike()]:
    v.start() # Different behavior, same method name
```

16.6 Method Resolution Order (MRO)

When a class has multiple parents, MRO determines which parent's method is called first. Python uses C3 linearization: **Child > Parent1 (left to right) > Parent2 > Object**.

Module 17: APIs (Application Programming Interface)

17.1 HTTP Methods

Sub-Topic	Explanation	Code Example
GET	Retrieve/extract data from an API.	<code>requests.get(url)</code>
POST	Create new data in the database.	<code>requests.post(url, data)</code>
PUT	Update an entire record.	<code>requests.put(url, data)</code>
PATCH	Update only specific fields.	<code>requests.patch(url, data)</code>
DELETE	Delete data from a table.	<code>requests.delete(url)</code>

17.2 HTTP Status Codes

Sub-Topic	Explanation	Code Example
1xx	Informational responses	100: Continue
2xx	Success	200: OK, 201: Created
3xx	Redirection	301: Moved Permanently
4xx	Client Error	400: Bad Request, 404: Not Found
5xx	Server Error	500: Internal Server Error

```
import requests
import json

response = requests.get('https://api.example.com/data')
if response.status_code == 200:
    data = response.json() # Parse JSON response
    print(data)
```



Capstone Project: Banking Application

The course culminates in building a complete banking application that integrates all concepts learned. This project demonstrates how Python fundamentals come together in a real-world application.

Features Implemented

Create Account

Generate random account numbers, store customer data in dictionaries.

View Balance

Access nested dictionary data, validate account existence with exception handling.

Deposit Money

Update dictionary values, validate inputs.

Withdraw Money

Conditional logic for balance checks, prevent overdraft.

Transfer Funds

Multi-account operations, validate both accounts, atomic transactions.

List Accounts

Loop through dictionary items, formatted output.

Concepts Applied

This project uses **functions, dictionaries, while loops, f-strings, conditional logic, user input, random module, and exception handling** — bringing together everything from Modules 1–12 into a cohesive application.

Interview Tip

Building projects like this banking app is exactly what interviewers look for. During **L1 interviews** (4-5 min Python section with 1 coding question), you may be asked to build a simplified version. For **L2** (manager round), you should be able to explain your project architecture and design decisions.